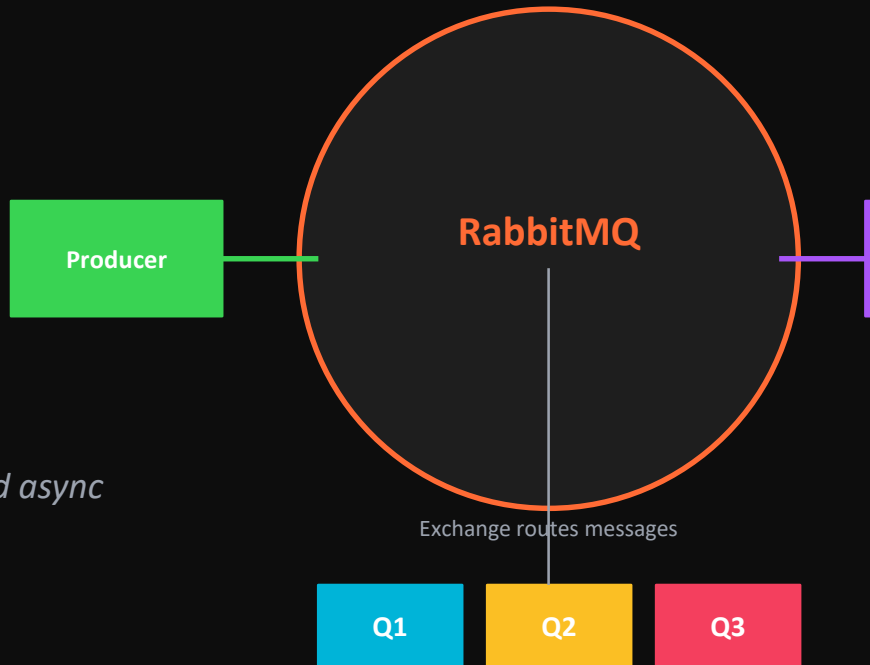


RabbitMQ

Deep Dive

Message brokers, exchanges, queues, and why your services need async communication



Producer

Exchange

Queue

Consumer

AMQP

What We'll Cover

01

What is RabbitMQ?

AMQP protocol, brokers, and why async messaging matters

02

Exchanges & Routing

Direct, Fanout, Topic, Headers — how messages get routed

03

Queues & Bindings

Durable queues, binding keys, message persistence

04

Reliability Patterns

Acknowledgments, dead-letter queues, publisher confirms

05

Clustering & HA

Mirrored queues, federation, high availability setup

06

RabbitMQ vs Kafka

When to use which — a comprehensive comparison

WHAT IS RABBITMQ?

The Message Broker

RabbitMQ decouples services by acting as an intermediary — producers send messages, consumers receive them asynchronously.



AMQP Protocol

Implements Advanced Message Queuing Protocol 0-9-1 for reliable messaging

Built with Erlang

Erlang's OTP platform gives RabbitMQ battle-tested concurrency and fault tolerance

Push-based

Broker pushes messages to consumers — no polling required

The 4 Exchange Types

Exchanges decide where messages go. Each type uses different routing logic.

DIRECT EXCHANGE

Routes by exact routing key match

Use: Task queues, point-to-point

e.g. routingKey: "order.paid" → only "order.paid" queue

FANOUT EXCHANGE

Broadcasts to ALL bound queues — ignores routing key

Use: Notifications, pub/sub, event broadcasting

e.g. 1 message → Q1, Q2, Q3, Q4 all receive it

TOPIC EXCHANGE

Routes via wildcard patterns (* = one word, # = many)

Use: Flexible multi-consumer routing

e.g. "logs..error" matches logs.app.error*

HEADERS EXCHANGE

Routes based on message header attributes

Use: Complex conditional routing without keys

e.g. { format: "pdf", type: "report" } → match

Durable, Persistent & Reliable

Durable

Queue survives broker restart — stored to disk

Exclusive

Only one consumer; deleted when connection closes

Auto-delete

Queue deleted when last consumer disconnects

Persistent msgs

delivery_mode=2 — message written to disk, survives restart

Dead Letter Queue (DLQ)

Message rejected or expired → routed to DLQ for inspection or retry. Set via **x-dead-letter-exchange** argument on the queue declaration.

Acknowledgments, Confirms & Retries

Consumer ACK

- `basicAck()` — success, remove from queue
- `basicNack()` — failure, requeue or discard
- `basicReject()` — reject single message
- Always ack or nack — never leave unacked

Prefetch / QoS

- `basicQos(prefetchCount=1)` — fair dispatch
- Limits unacknowledged msgs per consumer
- Prevents one fast consumer starving others
- Critical for load balancing in worker pools

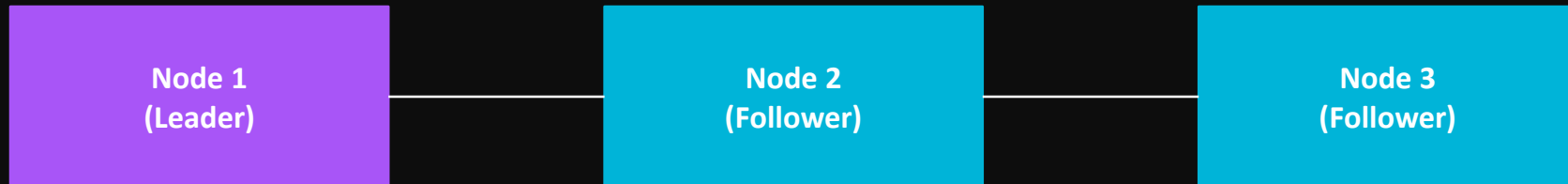
Publisher Confirms

- Channel in confirm mode: `channel.confirmSelect()`
- Broker ACKs when message is persisted
- NACKed messages should be retried
- Enables at-least-once delivery guarantee

TTL & Expiry

- Per-message TTL: `expiration` property
- Per-queue TTL: `x-message-ttl` argument
- Expired msgs → dead-letter exchange
- Queue TTL: `x-expires` (idle queue deletion)

Quorum Queues & Federation



Quorum Queues

Raft-based replication — replicas elect a leader. Majority must confirm write. Safe for critical data.

Classic Mirrored

Legacy: syncs all msgs to mirrors. High overhead — prefer Quorum Queues for new deployments.

Federation Plugin

Connects separate RabbitMQ brokers across data centers. Messages flow via AMQP links.

Shovel Plugin

Moves messages from source queue to destination, even across brokers. Good for migrations.

RabbitMQ vs Kafka

	 RabbitMQ	 Kafka
Model	Message Queue (push)	Log/Stream (pull)
Protocol	AMQP, STOMP, MQTT	Custom binary (TCP)
Throughput	20K–50K msg/sec	1M+ msg/sec
Message Retention	Deleted after ACK	Retained by time/size
Routing	Flexible (exchanges)	Topic partitions only
Ordering	Per-queue (single consumer)	Per-partition (strict)
Consumer Groups	Competing consumers	Consumer groups + replay
Best For	Task queues, RPC, complex routing	Event streaming, analytics

💡 Use RabbitMQ for task queues & smart routing | Use Kafka for high-throughput event streams & replay

When to Choose Which?

✓ Choose RabbitMQ when...

- You need complex routing (fanout, topic, headers)
- Building task queues with worker processes
- Need per-message TTL or dead-letter queues
- RPC-style request/reply patterns required
- Messages must be deleted after processing
- Lower message volume (<100K/sec)

⚡ Choose Kafka when...

- High throughput: millions of events/second
- Event replay / reprocessing history needed
- Multiple independent consumers of same stream
- Building event sourcing or CQRS architecture
- Log aggregation, metrics, clickstream analytics
- Messages need long-term retention (days/weeks)

Key Takeaways

RabbitMQ = Smart Broker

Routes messages intelligently via exchanges. Ideal for complex workflows with varied routing needs.

Reliability is Opt-In

Use durable queues + persistent messages + publisher confirms for guaranteed delivery.

RabbitMQ ≠ Kafka

Different tools for different jobs. RabbitMQ for routing & tasks; Kafka for streams & replay.

4 Exchange Types

Direct, Fanout, Topic, Headers — each serves a different routing pattern. Know when to use each.

Quorum Queues for HA

Replaces classic mirrored queues. Raft consensus ensures data safety in clusters.

Always ACK Messages

Unacknowledged messages stay in queue forever. Every consumer must ack, nack, or reject.